

Binary Search

Binary search is a searching algorithm used to find an element in a sorted array.

Instead of checking every element one by one, binary search compares the key with the middle element and eliminates half of the search space in each iteration.

The array must be sorted.

Working principle

1. Find the middle element of the array
2. Compare the middle element with the key
3. If they are equal return the Index.
4. If the key is greater than the middle element, search in the right half.
5. If the key is smaller than the middle element, search in the left half.
6. Repeat the process until the element is found or the search space becomes empty.

variables

Start = 0

end = n-1

$mid = start + (end - start) / 2;$

- Start → First index of the current search space
- end → last index of the current search space
- mid → middle index of the current search space.

Algorithm

- Step-1 $start = 0;$
 $end = n-1;$
- Step-2 $mid = start + (end - start) / 2$
- Step-3 if $arr[mid] == key;$
return mid;
- Step-4 if $key > arr[mid]$
search in right half
 $start = mid + 1;$
- Step-5 if $key < arr[mid]$
search in the left half
 $end = mid - 1;$
- Step-6 Recalculate the middle index
 $mid = start + (end - start) / 2;$
- Step-7 while loop
 $start <= end.$
return -1 (if the loop ends without finding the element, return)

Only Run

Array 2 4 6 8 12 18
key = 12

iteration 1

start = 0

end = 5

mid = 2

arr[mid] = 6

since $12 > 6$

update start = mid + 1;

Now

start = 3

end = 5

recalculate

mid = 4;

iteration 2

arr[mid] = 12

Element found

return 4;

2 Why is mid recalculated?

After updating start or end, the search range changes

Therefore, the previous middle index is no longer available valid.

A new middle index must be calculated for the updated search space

$$\text{mid} = \text{start} + (\text{end} - \text{start}) / 2;$$

3 Why do we use while (start <= end)

The loop continues as long as there are elements left to search.

if $\text{start} > \text{end}$

The search space becomes empty, which means the elements do not exist in the array.

Hence

return -1;

Programm:-

```
Int binarysearch (int arr[], int size, int key)
{
```

```
    int start = 0;
```

```
    int end = size - 1;
```

```
    int mid = start + (end - start) / 2;
```

```
    while (start <= end)
    {
```

```
        if (arr[mid] == key)
        {
```

```
            return mid;
        }
```

```
        if (key > arr[mid])
        {
```

```
            start = arr[mid] mid + 1;
```

```
        }
        else
        {
```

```
            end = mid - 1;
```

```
        }
        mid = start + (end - start) / 2;
```

```
    }
    return -1;
}
```